Heaven's light is our guide

Rajshahi University of Engineering &Technology

Department of Computer Science & Engineering

Lab Report

Title	**:** Implementing Design Patterns
Lab No.	: 02
Course Name	**:** Software Engineering Sessional
Course Code	**:** CSE 3206
Submission Date	**:** 11 August, 2026

Submitted by:	**Submitted to:**
Md. Mhafuz Taraq Farazi (2203031) Fatema Tuz Zohora (2203032) Abhishek Bhattacharjee (2203033) **Section:** A **Session:** 2022-2023	**Name:** Emrana Kabir Hashi **Designation:** Assistant Professor **Department:** CSE **Institution:** RUET

Software Engineering Lab Documentation: Cafe Order Management System

Project Overview

This project implements a single, unified software solution—a Cafe Order Management System—that integrates three fundamental Creational Design Patterns into one end-to-end workflow:

1. **Singleton Pattern**: Manages the central `OrderTracker` so order IDs remain unique and synchronized.
2. **Factory Method Pattern**: Manages `PaymentHandler` to dynamically instantiate and process payments (`CardPayment` vs. `CashPayment`).
3. **Abstract Factory Pattern**: Manages `PackagingFactory` to generate matching families of packaging (`ReusablePackagingFactory` vs. `DisposablePackagingFactory`).

1. Singleton Pattern (`OrderTracker`)

Pattern Category

Creational Design Pattern

Intent

Ensure a class has only one instance throughout the application lifecycle and provide a global point of access to that instance.

When to Use

- When a shared resource or counter (e.g., central order tracker, database connection) must be synchronized across the entire application.
- When duplicate instances would cause state divergence or duplicate IDs.

Real-World Applications in This Project

In our Cafe Order Management System, multiple terminals or cashiers may place orders simultaneously. The `OrderTracker` ensures that all orders increment a single, centralized order counter without producing duplicate order numbers.

Problem Scenario

If every cashier or order terminal creates its own new `OrderTracker` instance, each terminal starts counting from 1. Order #1 from Terminal A would collide with Order #1 from Terminal B.

The Singleton pattern guarantees that every terminal accesses the exact same instance in memory.

Participants

- **Singleton (`OrderTracker`)**: Stores the single instance in `_instance` and controls creation via `__new__`.
- **Client (`process_cafe_order`)**: Accesses the tracker to retrieve the next unique order ID.

UML Structure (Conceptual)

```
Cashier A --------\
                   +------> [ OrderTracker ] (Single Global Counter)
Cashier B --------/
```

Example Code (Python)

```
class OrderTracker:
    _instance = None  # Holds the unique instance

    def __new__(cls):
        # If no instance exists, create one; otherwise return existing
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance.order_count = 0
        return cls._instance

    def get_next_order_id(self) -> int:
        self.order_count += 1
        return self.order_count
```

Code Explanation

1. **`_instance`** is a class-level variable that holds the single shared object reference.
2. **`__new__()`** intercepts instance creation. On the first call, it initializes **`order_count = 0`**.
3. Subsequent calls return the existing object.
4. Calling **`get_next_order_id()`** safely increments the central counter for every order in the system.

Advantages

- Guarantees unique, non-colliding order IDs.
- Zero memory waste from redundant tracker objects.

Limitations

- Introduces global state which requires careful handling in concurrent/multi-threaded environments.

2. Factory Method Pattern (`PaymentHandler`)

Pattern Category

Creational Design Pattern

Intent

Define an interface for creating an object, but let subclasses decide which class to instantiate. Factory Method lets a class defer instantiation to subclasses.

When to Use

- When a system needs to process various payment methods without hardcoding specific payment classes into the core ordering logic.
- When you want to follow the **Open/Closed Principle** so new payment types (e.g., ApplePay, Crypto) can be added without modifying existing code.

Real-World Applications in This Project

In the cafe system, customers can pay using **Card** or **Cash**. The base **`PaymentHandler`** handles the billing workflow, while subclasses decide whether to instantiate **`CardPayment`** or **`CashPayment`**.

Problem Scenario

Without Factory Method, the checkout function would use hardcoded **`if/else`** checks:

```
# Poor design (Tight coupling):
if payment_type == "card":
    payment = CardPayment()
elif payment_type == "cash":
    payment = CashPayment()
```

Adding a new payment method requires modifying this core logic. The Factory Method pattern solves this by delegating creation to a dedicated factory method **`create_payment()`**.

Participants

- **Product (`Payment`)**: Abstract interface declaring **`pay(amount)`**.
- **Concrete Products (`CardPayment`, `CashPayment`)**: Implementations of the payment process.
- **Creator (`PaymentHandler`)**: Declares the factory method **`create_payment()`** and executes **`process_bill()`**.
- **Concrete Creators (`CardPaymentHandler`, `CashPaymentHandler`)**: Override **`create_payment()`** to return specific payment objects.

UML Structure (Conceptual)

```
PaymentHandler (Creator) ---------> Payment (Interface)
       ^                                    ^
       |                                    |
  +----+------------------+            +----+------------------+
  |                       |            |                       |
CardPaymentHandler  CashPaymentHandler  CardPayment        CashPayment
  | (creates)             | (creates)
  v                       v
CardPayment          CashPayment
```

Example Code (Python)

```
from abc import ABC, abstractmethod
```

```
# Product Interface
class Payment(ABC):
    @abstractmethod
    def pay(self, amount: float) -> None:
        pass

# Concrete Products
class CardPayment(Payment):
    def pay(self, amount: float) -> None:
        print(f"Paid ${amount:.2f} using Credit/Debit Card.")

class CashPayment(Payment):
    def pay(self, amount: float) -> None:
        print(f"Paid ${amount:.2f} using Cash.")

# Creator with Factory Method
class PaymentHandler(ABC):
    @abstractmethod
    def create_payment(self) -> Payment:
        """The Factory Method"""
        pass

    def process_bill(self, amount: float) -> None:
        payment = self.create_payment()
        payment.pay(amount)

# Concrete Creators
class CardPaymentHandler(PaymentHandler):
    def create_payment(self) -> Payment:
        return CardPayment()

class CashPaymentHandler(PaymentHandler):
    def create_payment(self) -> Payment:
        return CashPayment()
```

Code Explanation

1. **`Payment`** enforces that all payment types implement **`pay()`**.
2. **`PaymentHandler.process_bill()`** executes payment without knowing whether it is cash or card.
3. Subclasses decide which concrete payment object to instantiate via **`create_payment()`**.

Advantages

- Decouples the ordering workflow from concrete payment gateways.
- Open for extension: new payment types require only a new pair of classes.

Limitations

- Slightly increases the number of classes.

3. Abstract Factory Pattern (`PackagingFactory`)

Pattern Category

Creational Design Pattern

Intent

Provide an interface for creating **families of related or dependent objects** without specifying their concrete classes.

When to Use

- When an order requires a matching set of items (e.g., Cup + Container) that must belong to the same style or category (Reusable vs. Disposable).
- When you need to prevent incompatible product combinations.

Real-World Applications in This Project

- **Reusable Order**: Requires washable, reusable tableware (**`CeramicCup`** + **`GlassPlate`**).
- **Disposable Order**: Requires disposable packaging (**`PaperCup`** + **`PaperBag`**).

Problem Scenario

If the cashier independently creates individual packaging items, mistakes can occur:

```
cup = CeramicCup()       # Reusable item
bag = PaperBag()         # Disposable item (Incompatible combination!)
```

The Abstract Factory pattern guarantees that cups and containers are always created as matching pairs from the same family.

Participants

- **Abstract Products (`Cup`, `Container`)**: Interfaces for the packaging family.
- **Concrete Products (`CeramicCup`, `GlassPlate`, `PaperCup`, `PaperBag`)**: Specific items for each family.
- **Abstract Factory (`PackagingFactory`)**: Declares **`create_cup()`** and **`create_container()`**.
- **Concrete Factories (`ReusablePackagingFactory`, `DisposablePackagingFactory`)**: Produce matched sets of packaging.

UML Structure (Conceptual)

```
PackagingFactory (Interface) ---------------------> Cup (Interface) + Container
(Interface)
      ^                                                    ^                    ^
      |                                                    |                    |
  +---+--------------------+                               |                    |
  |                        |                               |                    |
ReusablePackagingFactory  DisposablePackagingFactory       |                    |
  |                        |                               |                    |
  +---> CeramicCup         +---> PaperCup -+---------------+                    |
  +---> GlassPlate         +---> PaperBag -+------------------------------------+
```

Example Code (Python)

```
from abc import ABC, abstractmethod
```

```
# Abstract Products
class Cup(ABC):
    @abstractmethod
    def serve(self) -> str:
        pass

class Container(ABC):
    @abstractmethod
    def serve(self) -> str:
        pass

# Reusable Family
class CeramicCup(Cup):
    def serve(self) -> str:
        return "Ceramic Cup (Reusable)"

class GlassPlate(Container):
    def serve(self) -> str:
        return "Glass Plate (Reusable)"

# Disposable Family
class PaperCup(Cup):
    def serve(self) -> str:
        return "Disposable Paper Cup (Disposable)"

class PaperBag(Container):
    def serve(self) -> str:
        return "Disposable Paper Bag (Disposable)"

# Abstract Factory Interface
class PackagingFactory(ABC):
    @abstractmethod
    def create_cup(self) -> Cup:
        pass

    @abstractmethod
    def create_container(self) -> Container:
        pass

# Concrete Factories
class ReusablePackagingFactory(PackagingFactory):
    def create_cup(self) -> Cup:
        return CeramicCup()

    def create_container(self) -> Container:
        return GlassPlate()

class DisposablePackagingFactory(PackagingFactory):
    def create_cup(self) -> Cup:
        return PaperCup()

    def create_container(self) -> Container:
        return PaperBag()
```

Code Explanation

1. **`PackagingFactory`** defines the interface for creating both a **`Cup`** and a **`Container`**.
2. **`ReusablePackagingFactory`** creates only reusable items (**`CeramicCup`** + **`GlassPlate`**).

3. **`DisposablePackagingFactory`** creates only disposable items (**`PaperCup`** + **`PaperBag`**).
4. The client receives guaranteed-compatible packaging elements.

Advantages

- Guarantees product compatibility across the packaging family.
- Eliminates the possibility of mixing Disposable and Reusable tableware.

4. Integrated System Demonstration

Unified Client Function

```
def process_cafe_order(
    item_name: str,
    amount: float,
    payment_handler: PaymentHandler,
    packaging_factory: PackagingFactory
) -> None:
    # 1. Singleton: Get next unique order ID
    tracker = OrderTracker()
    order_id = tracker.get_next_order_id()
    print(f">>> [Order #{order_id}] New Order: '{item_name}' (Total: ${amount:.2f})")

    # 2. Factory Method: Process payment
    payment_handler.process_bill(amount)

    # 3. Abstract Factory: Prepare matching packaging
    cup = packaging_factory.create_cup()
    container = packaging_factory.create_container()
    print(f"Packaging : Served in a {cup.serve()} & {container.serve()}")
    print(f"Order #{order_id} Completed Successfully!\n")
```

Live Execution Output

```
======================================================================
            CAFE ORDER MANAGEMENT SYSTEM
======================================================================

>>> [Order #1] New Order: 'Cappuccino' (Total: $8.50)
    Paid $8.50 using Credit/Debit Card.
    Packaging : Served in a Ceramic Cup (Reusable) & Glass Plate (Reusable)
    Order #1 Completed Successfully!

>>> [Order #2] New Order: 'Muffin' (Total: $6.75)
    Paid $6.75 using Cash.
    Packaging : Served in a Disposable Paper Cup (Disposable) & Disposable Paper Bag
(Disposable)
    Order #2 Completed Successfully!

>>> [Order #3] New Order: 'Double Espresso' (Total: $4.00)
    Paid $4.00 using Credit/Debit Card.
    Packaging : Served in a Ceramic Cup (Reusable) & Glass Plate (Reusable)
    Order #3 Completed Successfully!

Total Orders Processed Today (via Singleton): 3
======================================================================
```

5. Conclusion

By combining all three patterns into a single **Cafe Order Management System**:

- **Singleton (`OrderTracker`)** ensures global order synchronization.
- **Factory Method (`PaymentHandler`)** provides flexible payment processing.
- **Abstract Factory (`PackagingFactory`)** enforces consistent product families.

The resulting architecture is modular, extensible, loosely coupled, and adheres to standard Object-Oriented Design principles.